

Docflow

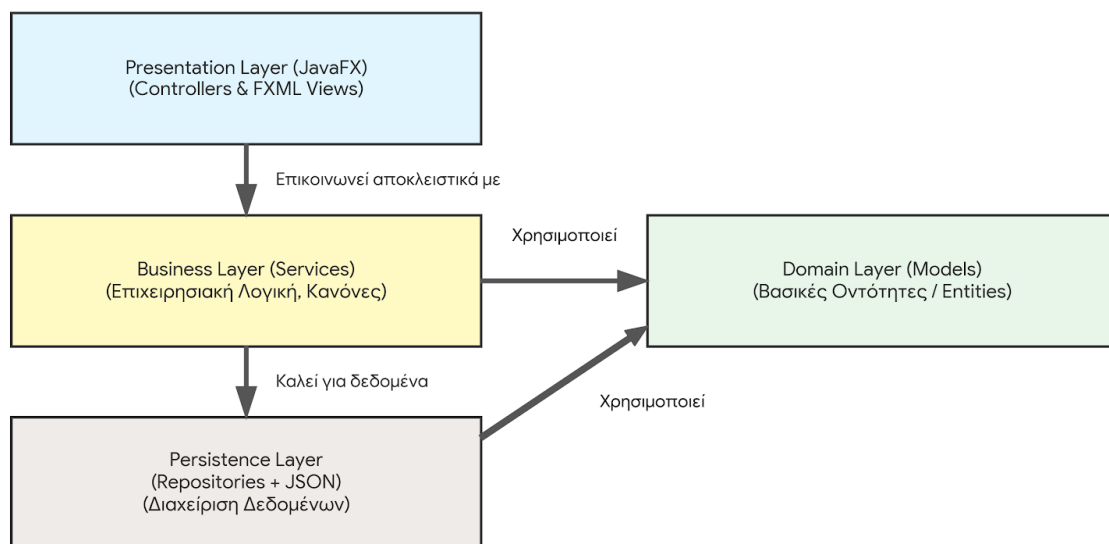
Το **Docflow** είναι μια εφαρμογή διαχείρισης εγγράφων με υποστήριξη πολλαπλών χρηστών με διαφορετικούς ρόλους και διαφοροποιημένα δικαιώματα πρόσβασης. Υλοποιήθηκε σε **Java 17**, με χρήση **JavaFX** (controls, FXML) για το frontend και **Maven** για τη διαχείριση του build και των εξαρτήσεων.

Τα δεδομένα του συστήματος αποθηκεύονται σε αρχεία **JSON** στον φάκελο medialab/ [users.json, documents.json, categories.json]. Η διαχείριση της ανάγνωσης και εγγραφής των JSON δεδομένων στα αρχεία υλοποιείται με τη βιβλιοθήκη **Gson**.

Κατά την εκκίνηση της εφαρμογής φορτώνονται όλα τα δεδομένα στη μνήμη και δημιουργούνται τα αντίστοιχα αντικείμενα. Κατά την εκτέλεση, όλες οι λειτουργίες πραγματοποιούνται αποκλειστικά πάνω στα **in-memory αντικείμενα**. Η ενημέρωση των JSON αρχείων γίνεται μόνο στον τερματισμό της εφαρμογής.

Το σύστημα υποστηρίζει **versioning** για τα έγγραφα, όπου κάθε τροποποίηση δημιουργεί νέα έκδοση χωρίς διαγραφή των προηγούμενων, καθώς και role-based έλεγχο δικαιωμάτων. Για την υποστήριξη αυτών των απαιτήσεων επιλέχθηκε αρχιτεκτονική 4 επιπέδων (**Layered Architecture**), η οποία περιγράφεται στη συνέχεια:

Αρχιτεκτονική 4 Επιπέδων (Layered Architecture)



Presentation Layer (Controllers, FXML): Διαχειρίζεται αποκλειστικά τη γραφική διεπαφή μέσω FXML και JavaFX Controllers. Χειρίζεται τα UI events και προωθεί τα αιτήματα στα Services, διατηρώντας το UI ανεξάρτητο από τη λογική των δεδομένων.

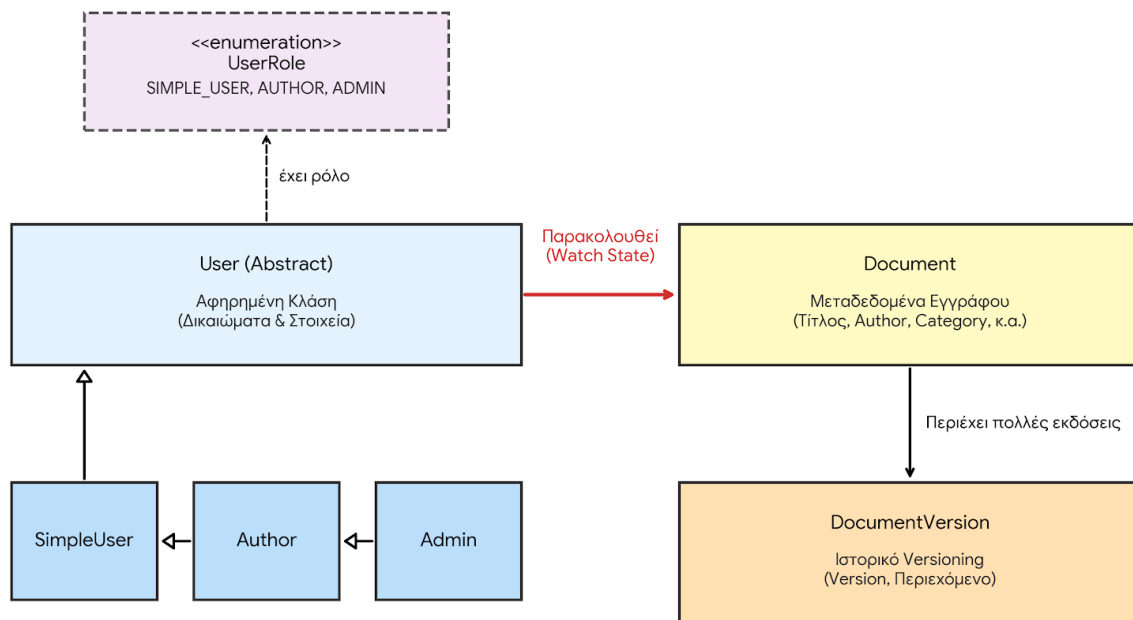
Business Layer (Services): Υλοποιεί τους κανόνες του συστήματος, όπως έλεγχο δικαιωμάτων, versioning και διαχείριση εγγράφων. Δέχεται αιτήματα από το UI και συντονίζει τις απαραίτητες ενέργειες πάνω στα domain objects.

Domain Layer (Models): Περιλαμβάνει τις βασικές οντότητες του συστήματος (User, Document, Category κ.α.) και εκφράζει τη δομή και τη συμπεριφορά τους. Τα models δεν περιέχουν λογική διεπαφής ή αποθήκευσης.

Persistence Layer (Repositories): Είναι υπεύθυνο για τη μόνιμη αποθήκευση και ανάκτηση των δεδομένων από τα αρχεία JSON (φάκελος medialab/). Απομονώνει πλήρως τις λειτουργίες load/save και δεν περιέχει επιχειρησιακή λογική.

Η διαχείριση της συνολικής κατάστασης της εφαρμογής υλοποιείται μέσω της κλάσης **AppState**, η οποία λειτουργεί ως κεντρικό σημείο συντονισμού. Δημιουργεί και συνδέει τα repositories και τα services, διατηρώντας την κοινή κατάσταση του συστήματος.

Διάγραμμα Οντοτήτων / Domain Model



JSON Data Schema

users.json

```
[
  {
    "firstName": "string",
    "lastName": "string",
    "username": "string",
    "password": "string",
    "role": "SIMPLE_USER | AUTHOR | ADMIN",
    "allowedCategories": [
      "string"
    ],
    "followedDocuments": [
      "documentId-string"
    ],
    "lastSeenVersions": {
      "documentId-string": 0
    }
  }
]
```

documents.json

```
[
  {
    "id": "uuid-string",
    "title": "string",
    "author": "string",
    "category": "string",
    "createdAt": "YYYY-MM-DD",
    "modifiedAt": "YYYY-MM-DD",
    "versions": [
      {
        "versionNumber": 1,
        "content": "string"
      }
    ]
  }
]
```

categories.json

```
[
  "string"
]
```

Τα αναγνωριστικά των εγγράφων υλοποιούνται ως **UUID (String)**, ώστε να διασφαλίζεται μοναδικότητα χωρίς χρήση αύξοντων τιμών και χωρίς εξάρτηση από τη σειρά δημιουργίας.

Για τους users χρησιμοποιείται ως μοναδικό αναγνωριστικό το **username** (case-sensitive), ενώ για τα categories το **name**. Με τον τρόπο αυτό επιβάλλεται μοναδικότητα σε επίπεδο domain και αποτρέπονται διπλότυπες εγγραφές.

Παραδοχές και Σχεδιαστικές Επιλογές

- Πολλές βασικές ενέργειες υποστηρίζονται μέσω **context menus** ή **διπλού click** στους πίνακες, για ταχύτερη πλοήγηση.
- Η επιλογή **“Admin Settings”** εμφανίζεται μόνο όταν ο συνδεδεμένος χρήστης έχει ρόλο Admin.
- Ανάλογα με τον ρόλο του χρήστη, ορισμένες επιλογές της διεπαφής είτε δεν εμφανίζονται είτε εμφανίζονται σε disabled κατάσταση.
- Κάθε Admin έχει πάντοτε πρόσβαση σε όλες τις κατηγορίες, ανεξαρτήτως ρυθμίσεων allowedCategories.
- Σε περίπτωση διαγραφής χρήστη, τα έγγραφα που έχει δημιουργήσει παραμένουν στο σύστημα και δεν διαγράφονται.
- Σε περίπτωση διαγραφής κατηγορίας, διαγράφονται αυτόματα όλα τα έγγραφα που ανήκουν σε αυτήν.
- Η επεξεργασία χρήστη από τον διαχειριστή περιορίζεται στα πεδία firstName, lastName και allowedCategories. Τα username, role και password θεωρούνται μη επεξεργάσιμα μετά τη δημιουργία του χρήστη.
- Ο προεπιλεγμένος διαχειριστής (medialab) είναι hardcoded στον κώδικα και δημιουργείται ακόμη και αν δεν υπάρχει σχετική εγγραφή στα JSON αρχεία.
- Ένας admin δεν μπορεί να διαγράψει τον εαυτό του, ενώ ο default admin δεν μπορεί να διαγραφεί από κανέναν άλλον admin.
- Η ειδοποίηση για νέα έκδοση παρακολουθούμενου εγγράφου παραμένει ενεργή έως ότου ο χρήστης ανοίξει το συγκεκριμένο έγγραφο.
- Η αναζήτηση σε Title και Author υλοποιείται με partial match (case-insensitive contains), ενώ στο Category με exact match.
- Για την ευκολία στο development υπάρχει επιλογή **AUTO_LOGIN_AS_ADMIN=true** στον κώδικα για αυτόματη σύνδεση ως admin.

Αξιοσημείωτες Επιπλέον Λειτουργίες και Λεπτομέρειες στην Εφαρμογή

- Κουμπί αντιγραφής περιεχομένου εγγράφου στο clipboard (**Copy**).
- Κουμπί επικόλλησης κειμένου από το clipboard στη δημιουργία νέου εγγράφου (**Paste clipboard**).
- Μετρητής λέξεων στα dialogs δημιουργίας και επεξεργασίας/προβολής εγγράφου.
- Επιλογή παλαιότερης έκδοσης και **restore** του περιεχομένου της ως νέο draft.
- Ένδειξη unsaved changes (*) στον τίτλο παραθύρου και επιβεβαίωση πριν το κλείσιμο.
- Ξεχωριστό **“Document Info”** dialog για προβολή μεταδεδομένων εγγράφου και counters (words, characters, paragraphs) της τρέχουσας έκδοσης.
- Ένδειξη κατάστασης ενημέρωσης ανά έγγραφο (π.χ. **“Update: v3 → v4”**, **“Following: No updates”**).
- **Show/Hide password** στο login και στη φόρμα δημιουργίας/επεξεργασίας χρήστη.
- Ένδειξη πλήθους αποτελεσμάτων στο κάτω μέρος των πινάκων, με μορφή **“X of Y Documents”** όταν εφαρμόζονται φίλτρα.

Σχεδιασμός Γραφικής Διεπαφής (GUI)

Η γραφική διεπαφή υλοποιήθηκε με τα προεπιλεγμένα **JavaFX components** και **custom CSS**, με minimal σχεδιασμό εμπνευσμένο από σύγχρονα UI systems (π.χ. [shadcn/ui](#)). Τα διάφορα εικονίδια προέρχονται από το [Lucide](#), συμβάλλοντας σε συνεπές και καθαρό οπτικό αποτέλεσμα.